Relational and Non-Relational Database Design:

A Case Study of Amazon

Introduction

Amazon Store (https://www.amazon.co.uk) was selected as a case study to explore both relational (SQL) and non-relational (MongoDB) database design.

As a large e-commerce platform, Amazon manages a huge volume of data, including customers, products, and orders. This makes it a suitable example for analysing how different database systems handle structured data, scalability, and performance.

This project compares relational and non-relational approaches and demonstrates how each model can be used to support the needs of a global online store.

I. Relational Design

1. Database Design Process

To begin the database design process, it is important to define what data needs to be stored, how it will be accessed, relevant business rules, and key use cases.

1.1. Data to be stored

Amazon needs to store customer information (e.g. name, email, address, order history), as well as product data such as pricing, stock levels, and seller details.

1.2. Data access requirements

Access to data must be controlled. Users should only access their own data through secure authentication. Sensitive information, such as payment details, must be encrypted. Internal access should follow role-based access control.

1.3. Business rules

Amazon must follow legal regulations such as GDPR, which require secure storage and proper handling of personal data. Transaction data must be retained for auditing and fraud prevention. Pricing and availability may vary by location.

1.4. Use cases

The data supports key functions such as order processing, delivery tracking, and customer support. It is also used for recommendations, personalised offers, and business analytics, including demand forecasting and inventory management.

2. Main database entities

The Amazon database includes several main entities:

- Customers (customer_id, name, contact details, registration date),
- Products (product_id, category_id, seller_id, product details, price, stock),
- Orders (order_id, customer_id, order date, status, total amount),
- Order_Items (order_id, product_id, quantity, price),
- as well as supporting entities such as Categories, Sellers, Deliveries, Payments, and Reviews, etc.

Three key entities are analysed in this project: **Customers, Products, and Orders.**

ER – diagram

The ER diagram illustrates the relationships between the main entities.

There is a one-to-many relationship between Customers and Orders, where one customer can place multiple orders.

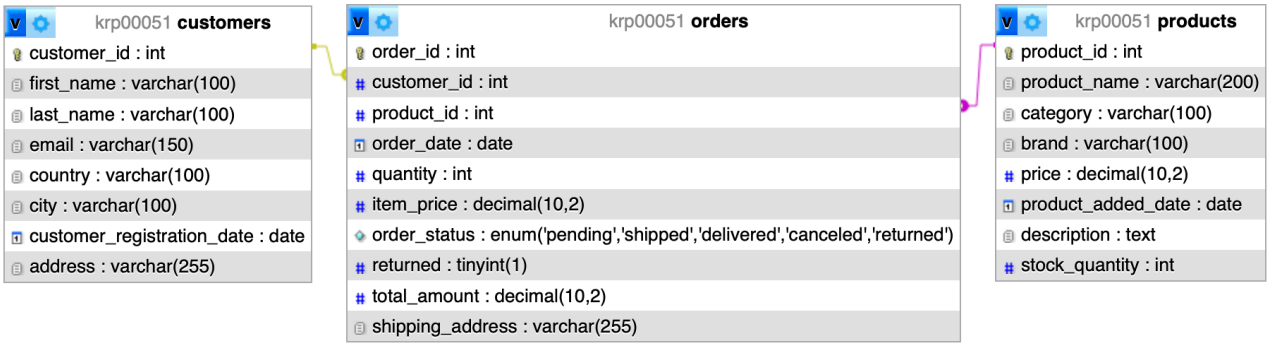
There is also a relationship between Orders and Products, where each order is linked to a single product through the product_id foreign key.

In this simplified model, each order contains only one product. However, in a full relational database design, this would normally be represented as a many-to-many relationship using a separate Order_Items table.

Database Schema

The presented database design satisfies the First and Second Normal Forms (1NF and 2NF), as all attributes are atomic and depend on the primary key.

However, it only partially satisfies the Third Normal Form (3NF), because the Orders table stores product-related attributes such as product_id, quantity, and item_price.




```sql
CREATE TABLE Customers (
    customer_id INT
PRIMARY KEY AUTO_INCREMENT,
    first_name VARCHAR(100) NOT NULL,
    last_name VARCHAR(100) NOT NULL,
    email VARCHAR(150)
NOT NULL UNIQUE,
    country VARCHAR(100)
NOT NULL,
    city VARCHAR(100)
NOT NULL,
    address VARCHAR(255)
NOT NULL,
    customer_registration_date DATE NOT NULL
)
ENGINE=InnoDB DEFAULT CHARSET=utf8;
```

```sql
CREATE TABLE Orders (
    order_id INT PRIMARY KEY AUTO_INCREMENT,
    customer_id INT NOT NULL,
    product_id INT NOT NULL,
    order_date DATE NOT NULL,
    quantity INT NOT NULL,
    item_price DECIMAL(10,2) NOT NULL,
    order_status ENUM('pending','shipped','delivered',
                      'canceled','returned')
    NOT NULL DEFAULT 'pending',
    returned TINYINT(1) NOT NULL DEFAULT 0,
    total_amount DECIMAL(10,2) NOT NULL,
    shipping_address VARCHAR(255) NOT NULL,
    CONSTRAINT fk_orders_customer FOREIGN KEY (customer_id)
    REFERENCES Customers(customer_id),
    CONSTRAINT fk_orders_product FOREIGN KEY (product_id)
    REFERENCES Products(product_id)
)
ENGINE=InnoDB DEFAULT CHARSET=utf8;
```

```sql
CREATE TABLE Products (
    product_id INT
PRIMARY KEY AUTO_INCREMENT,
    product_name VARCHAR(200) NOT NULL,
    category VARCHAR(100) NOT NULL,
    brand VARCHAR(100),
    price DECIMAL(10,2)
NOT NULL,
    product_added_date DATE NOT NULL,
    description TEXT
NOT NULL,
    stock_quantity INT
NOT NULL
)
ENGINE=InnoDB DEFAULT CHARSET=utf8;
```

The Orders table contains its own primary key (order_id) and two foreign keys: customer_id referencing Customers and product_id referencing Products.

Address data is also simplified: the Customers table stores a general address, while the Orders table contains a shipping_address, since delivery details may differ for each order. This simplification improves clarity but limits the flexibility of representing multiple products per order.

Data integrity in this database is maintained using constraints such as primary keys, foreign keys, NOT NULL, UNIQUE, and ENUM.

- *Entity integrity* is ensured by primary keys, which prevent duplicate records. Referential integrity is maintained through foreign keys in the Orders table, linking Customers and Products and ensuring that each order is associated with valid entities.

- *Domain integrity* is supported by appropriate data types and constraints. NOT NULL is applied to essential attributes to prevent missing data, while the UNIQUE constraint ensures that each customer email is distinct.

- *The ENUM constraint* is used for the order_status attribute (pending, shipped, delivered, cancelled, returned), ensuring consistent values.

- *DEFAULT* is also applied to improve consistency and reduce manual input.

Together, these constraints ensure data accuracy, consistency, and reliability.

3. SQL Queries

3.1. Number of customers registered per month by country

This query helps analyse customer growth across different countries over time. An increase in new customers indicates successful marketing and business expansion, while a decrease may suggest potential issues such as competition or ineffective campaigns.

By comparing countries, Amazon can identify fast-growing markets and make strategic decisions, such as adjusting marketing efforts or expanding logistics and infrastructure.

This type of analysis is important for understanding market trends and planning future growth.

```sql
SELECT country,
       YEAR(customer_registration_date) AS year,
       MONTH(customer_registration_date) AS month,
       COUNT(*) AS new_customers
FROM customers
GROUP BY country, year, month
ORDER BY year, month, country;
```

3.2. Products with the highest sales and lowest return rates

This query identifies products with high sales and low return rates. These products indicate strong customer satisfaction and generate higher profit with lower return costs.

Such products should be prioritised for promotion, stock management, and recommendation systems to maximise revenue.

This helps Amazon focus on high-performing products and improve overall efficiency.

```
SELECT products.product_name,
       SUM(orders.quantity) AS total_sales,
       SUM(CASE WHEN orders.returned = 1 THEN 1 ELSE 0 END) AS total_returns
FROM orders
JOIN products
  ON orders.product_id = products.product_id
GROUP BY products.product_id, products.product_name
ORDER BY total_sales DESC, total_returns ASC;
```

3.3. Customers with the highest spending over the last 3 months

This query identifies customers who have spent the most money over the last three months. These customers are valuable to the business, as they are more likely to make repeat purchases and contribute significantly to total revenue.

Identifying high-value customers supports targeted marketing strategies, such as personalised offers, loyalty programmes, and retention campaigns. It also helps detect changes in customer behaviour, including potential churn, allowing the company to take appropriate actions.

```
SELECT customers.customer_id,
       customers.first_name,
       customers.last_name,
       SUM(orders.total_amount) AS total_spent
FROM customers
JOIN orders
  ON customers.customer_id = orders.customer_id
WHERE orders.order_date >= DATE_SUB(CURDATE(), INTERVAL 3 MONTH)
GROUP BY customers.customer_id, customers.first_name, customers.last_name
ORDER BY total_spent DESC
LIMIT 30;
```

II. NoSQL Design

1. Limitations of relational databases and how MongoDB solves them

Relational databases use a fixed schema, which makes it difficult to handle diverse data such as different product types on Amazon (e.g. electronics, clothing, food). They also rely on JOIN operations to combine data from different tables, which can become slow with large datasets. In addition, relational databases do not scale horizontally easily, which is a limitation for global platforms with millions of users.

MongoDB addresses these limitations by using a flexible document-based model. It allows data with different structures to be stored without modifying the schema. By embedding related data, it reduces the need for JOIN operations and improves performance.

MongoDB also supports horizontal scaling through sharding, allowing data to be distributed across multiple servers and reducing latency for users.

2. Replacing relational elements (foreign keys, normalisation)

In MongoDB, the system can be represented using the same three collections: **Customers, Products, and Orders**, following the principle that data which is accessed together should be stored together.

In relational databases, foreign keys are used to link tables. In MongoDB, this can be replaced by embedding (e.g. product details inside orders) and referencing (e.g. storing customer_id in orders).

Unlike relational databases, MongoDB often uses denormalisation, where some data is duplicated to improve read performance.

When distributing the database across a cluster, several key decisions must be made, including choosing an appropriate shard key (e.g. region or customer_id), using replication to ensure fault tolerance and high availability, and balancing consistency and performance.

For example, strong consistency may be required for payments, orders, and stock updates, while eventual consistency is required for features such as recommendations and reviews.

A Primary/Secondary architecture can also be used in MongoDB, where the primary node handles write operations and secondary nodes replicate the data, ensuring system availability and reliability.

In MongoDB, the collections store data as flexible JSON-like documents. Constraints such as UNIQUE can be implemented using indexes. MongoDB automatically generates a unique _id for each document instead of using an auto-increment primary key.

Customers Collection in MongoDB:

```
db.customers.insertOne({
  first_name: "Kristina",
  last_name: "Palesheva",
  email: "kristina@email.com",
  country: "UK",
  city: "Stirling",
  address: "12 High Street",
  customer_registration_date: new Date("2026-04-06")})
# Create a UNIQUE index to ensure emails are not duplicated
db.customers.createIndex(
  { email: 1 },
  { unique: true })
```

```
_id: ObjectId('69d3a0e9fc35672c10aea6b2'),
first_name: 'Kristina',
last_name: 'Palesheva',
email: 'kristina@email.com',
country: 'UK',
city: 'Stirling',
address: '12 High Street',
customer_registration_date: ISODate('2026-04-06T00:00:00.000Z')
```

Products Collection in MongoDB:

```
db.products.insertOne({
  product_name: "Laptop",
  category: "Electronics",
  brand: "Dell",
  price: 1200,
  product_added_date: new Date("2026-01-01"),
  description: "No description available",
  stock_quantity: 10})
```

```
_id: ObjectId('69d3a8c8fc35672c10aea6b5'),
product_name: 'Laptop',
category: 'Electronics',
brand: 'Dell',
price: 1200,
product_added_date: ISODate('2026-01-01T00:00:00.000Z'),
description: 'No description available',
stock_quantity: 10
```

Orders Collection in MongoDB:

```
db.orders.insertOne({
  customer_id: ObjectId("69d3a0e9fc35672c10aea6b2"),
  customer: {
    name: "Alice Brown",
    country: "UK"},
  order_date: new Date("2025-12-15"),
  order_status: "delivered",
  shipping_address: {
    street: "12 High Street",
    city: "Stirling",
    country: "UK"},
  total_amount: 200,
  items: [
    {product_id: ObjectId("70a123456789abcdef333333"),
      product_name: "Laptop Bag",
      category: "Accessories",
      brand: "HP",
      price: 150,
      quantity: 1,
      returned: true},
    {product_id: ObjectId("70a123456789abcdef222222"),
      product_name: "Mouse",
      category: "Electronics",
      brand: "Logitech",
      price: 50,
      quantity: 1,
      returned: false}]})
```

```
{
  _id: ObjectId('69d508d8fc35672c10aea6d6'),
  customer_id: ObjectId('69d3a0e9fc35672c10aea6b2'),
  customer: { name: 'Alice Brown', country: 'UK' },
  order_date: ISODate('2025-12-15T00:00:00.000Z'),
  order_status: 'delivered',
  shipping_address: { street: '12 High Street', city: 'Stirling', country: 'UK' },
  total_amount: 200,
  items: [
    {
      product_id: ObjectId('70a123456789abcdef333333'),
      product_name: 'Laptop Bag',
      category: 'Accessories',
      brand: 'HP',
      price: 150,
      quantity: 1,
      returned: true
    },
    {
      product_id: ObjectId('70a123456789abcdef222222'),
      product_name: 'Mouse',
      category: 'Electronics',
      brand: 'Logitech',
      price: 50,
      quantity: 1,
      returned: false
    }
  ]
},
```

In the **Orders collection**, customer data is represented using a reference (customer_id), while product information is embedded directly within the order document. This approach eliminates the need for JOIN operations and improves read performance.

It also preserves a snapshot of product data at the time of purchase, ensuring an accurate order history. If a product price changes later, it is updated in the Products collection, while the original price remains unchanged in the Orders collection.

In addition, each order contains an embedded array of products, which better reflects real e-commerce transactions where customers often purchase multiple items in a single order. In a relational database, this would require an additional table (e.g. Order_Items) and multiple JOIN operations, whereas in MongoDB this structure can be represented naturally within a single document.

3. Three Analytical Queries in MongoDB

3.1. Identify products that are frequently bought together

This query identifies pairs of products that are frequently purchased within the same order. *It supports recommendation systems such as "Frequently Bought Together" and helps Amazon create product bundles and improve cross-selling strategies.*

```
db.orders.aggregate([
  { $unwind: "$products" },
  {$group: {_id: "$_id",
     products: { $push: "$products.product_name" }}},
  {$project: {pairs: {$reduce: {input: "$products",initialValue: [],
         in: {$concatArrays: ["$$value",{$map: {input: "$products", as: "p2",
                 in: {product1: "$$this", product2: "$$p2"}}}]}}}}},
  { $unwind: "$pairs" },
  {$match: {
     $expr: { $lt: ["$pairs.product1", "$pairs.product2"] }}},
  {$group: {_id: {product1: "$pairs.product1",
       product2: "$pairs.product2"},
     frequency: { $sum: 1 }}},
  { $sort: { frequency: -1 } }])
```

```
{ _id: { product1: 'Laptop', product2: 'Mouse' }, frequency: 2 },
{ _id: { product1: 'Keyboard', product2: 'Laptop Bag' }, frequency: 1 },
{ _id: { product1: 'Laptop Bag', product2: 'Mouse' }, frequency: 1 }
```

3.2. Identify high-value customers at risk of churn

This query identifies customers who previously spent significant amounts but have not made recent purchases. It supports customer retention strategies by detecting potential churn.

By analysing order history within a single collection, Amazon can target these customers with personalised offers, discounts, or reminders to encourage them to return and continue purchasing.

```
db.orders.aggregate([
 {$group: {_id: "$customer_id",
    customer_name: { $first: "$customer.name" },
    customer_country: { $first: "$customer.country" },
    last_order: { $max: "$order_date" },
    total_spent: { $sum: "$total_amount" },
    total_orders: { $sum: 1 }}},
 {$addFields: {days_since_last_order: {divide: [
        { $subtract: [ISODate("2026-04-10"), "$last_order"] },1000 * 60 * 60 * 24] }}},
 {$match: {total_spent: { $gt: 500 },
    days_since_last_order: { $gt: 60 }}}, {$sort: { total_spent: -1 }}])
```

```
_id: ObjectId('dddddddddddddddddddddddd'),
customer_name: 'David Smith',
customer_country: 'UK',
last_order: ISODate('2025-12-05T00:00:00.000Z'),
total_spent: 1100,
total_orders: 2,
days_since_last_order: 126
```

3.3. Analyse revenue and return rate by product category

This query analyses total revenue and return rates across different product categories. It helps Amazon identify which categories are most profitable and which generate higher return costs.

This information supports inventory planning, product quality improvement, and strategic decision-making, such as prioritising high-performing categories and addressing issues in categories with high return rates.

```
db.orders.aggregate([
  { $unwind: "$products" },
  {$group: {_id: "$products.category", total_revenue: {
        $sum: {$multiply: ["$products.price", "$products.quantity"]}},
      total_quantity_sold: {$sum: "$products.quantity"},
      total_returned_items: {
        $sum: {$cond: {if: "$products.returned", then: "$products.quantity", else: 0}}}}},
  {$addFields: {return_rate: {
        $cond: {if: { $eq: ["$total_quantity_sold", 0] }, then: 0,
          else: {$divide: ["$total_returned_items", "$total_quantity_sold"]}]}}}},
  {$sort: { total_revenue: -1 }}])
```

```
_id: 'Electronics',
total_revenue: 2400,
total_quantity_sold: 8,
total_returned_items: 2,
return_rate: 0.25
```

Conclusion

Both relational (SQL) and non-relational (MongoDB) databases were analysed using an Amazon store as an example.

SQL databases are very structured, which makes the data easier to understand and organise. They work well when relationships between data are clear and consistency is important.

However, when the system grows rapidly, SQL databases become unsufficient. Large platforms like Amazon need to handle a lot of data at the same time, so scalability becomes a key requirement.

MongoDB helps to solve this problem by using a distributed system, where data can be stored across multiple servers. This makes the system faster, more reliable, and more available. It also allows more flexible data structures, which is useful for things like products that have different attributes.

Overall, both approaches are useful and in real systems, using a combination of both can give the best results.